

Dormitory Management System — Design Compliance Report

Project: Student Dormitory Management System
Module: Advanced Web Development — University of Hildesheim
Reference Document: Final Design.pdf (submitted 2026-01-27)
Report Date: 2026-03-15

Overview

This report compares the implemented Dormitory Management System against the Final Design submission. Each design artifact — Angular architecture, data model, behavioral design, and backend design — is assessed for compliance. Items are classified as Implemented, Partially Implemented, or Not Implemented, with justification drawn from the actual codebase.

1. Angular Architecture — Components, Services & Modules

1.1 Components

The Final Design specifies nine distinct Angular components responsible for managing the user interface: AppComponent, NavMenuComponent, LoginComponent, StudentDashboardComponent, AdminDashboardComponent, ApplicationComponent, ContractComponent, ComplaintComponent, DocumentsComponent, ReceiptComponent, ProgressComponent.

Designed Component	Status	Implementation Notes
AppComponent	Implemented	src/app/app.ts — root component with RouterOutlet shell. Matches design intent.
LoginComponent	Implemented	src/app/pages/login/ — authentication form calling AuthService. Matches design.
StudentDashboardComponent	Implemented	src/app/pages/student-dashboard/ — student self-service dashboard. Present and functional.
AdminDashboardComponent	Implemented	src/app/pages/dashboard/ — admin dashboard. Present and functional.
NavMenuComponent	Implemented	src/app/components/nav-menu/nav-menu.ts — standalone presentational component with @Input() navItems, @Input() activeNav, @Output() navSelect, and @Output() sidebarToggle. Renders a <nav> list with active-state binding.
ApplicationComponent	Implemented	src/app/components/application/application.ts — standalone component accepting @Input() applications, @Input() loading, @Input() error, and emitting @Output() submitApplication, @Output() uploadFiles, @Output() updateStatus. Renders application list with status badges.

ContractComponent	Implemented	src/app/components/contract/contract.ts — standalone presentational component handling contract display and signed-contract upload events via @Output().
ComplaintComponent	Implemented	src/app/components/complaint/complaint.ts — standalone component displaying complaint history and emitting @Output() submitComplaint.
DocumentsComponent	Implemented	src/app/components/documents/documents.ts — standalone component displaying file list and emitting @Output() downloadFile.
ReceiptComponent	Implemented	src/app/components/receipt/receipt.ts — standalone component with payment form and @Output() submitReceipt.
ProgressComponent	Implemented	src/app/components/progress/progress.ts — standalone component bound to @Input() percentage, @Input() status, @Output() cancelProgress, and @Output() download. Renders the live progress bar modal.
RegisterComponent (not in design)	Added	src/app/pages/register/ — full student registration UI backed by POST /api/register. Not in design; added correctly.
DashboardSectionComponent (not in design)	Added	Empty stub component for Angular child routes under /dashboard/ and /student-dashboard/. Not in design; added to support router-based navigation.

Summary:All 11 designed components are implemented. The 7 feature components (NavMenuComponent, ApplicationComponent, ContractComponent, ComplaintComponent, DocumentsComponent, ReceiptComponent, ProgressComponent) follow the smart/dumb component pattern — they exist as standalone presentational components with @Input/@Output bindings, while orchestration logic lives in the two dashboard components.

1.2 Business Services

The Final Design specifies seven domain-specific services encapsulating business logic: AuthenticationService, ApplicationService, ContractService, ComplaintService, ReceiptService, FileService, ReportService, ProgressService.

Designed Service	Status	Implementation Notes
Authentication Service	Partially Implemented	AuthService in src/app/services/auth.service.ts handles login, logout, JWT token storage, and role helpers (isAdmin(), isStudent(), getRole()). Missing: HTTP interceptor pattern for token injection (headers are set manually in ApiService).
ApplicationService	Implemented	src/app/services/application.service.ts — delegates to ApiService. Exposes: getApplications(), submitApplication(), uploadApplicationFiles(), getApplicationFiles(), updateApplicationStatus().
ContractService	Implemented	src/app/services/contract.service.ts — delegates to ApiService. Exposes contract retrieval, upload, download, and status update methods.
ComplaintService	Implemented	src/app/services/complaint.service.ts — delegates to ApiService. Exposes getComplaints(), submitComplaint(), updateComplaintStatus().
ReceiptService	Implemented	src/app/services/receipt.service.ts — delegates to ApiService. Exposes payment recording, retrieval, verification, and receipt download methods.
FileService	Implemented	src/app/services/file.service.ts — delegates to ApiService. Exposes file upload, download, and metadata retrieval.
ReportService	Implemented	src/app/services/report.service.ts — delegates to ApiService. Exposes report generation, progress retrieval, cancellation, and download.

ProgressService	Implemented	src/app/services/progress.service.ts — delegates to ApiService.getReportProgress(). Owns startPolling() (creates a setInterval that calls the API every 600ms) and stopPolling() (calls clearInterval). Isolates polling logic from dashboard components.
ApiService (not in design)	Added	src/app/services/api.service.ts — monolithic service consolidating all 30+ direct API calls. Domain services delegate to this class. Not in design, which specified separate domain services; acts as an internal HTTP adapter layer.
NotificationService (not in design)	Added	src/app/services/notification.service.ts — handles SSE connection, bell notifications, and toast pop-ups. Not in design; added as an extra feature.

Summary: 7 of 8 designed services fully implemented; 1 (AuthService) partially implemented. The domain services delegate to ApiService rather than directly issuing HTTP calls — this is a valid facade/adaptor pattern that satisfies the design's intent of named domain boundaries, though the HTTP implementation detail lives in ApiService.

1.3 Infrastructure Services (Route Guards)

The Final Design explicitly specifies two infrastructure guards: AuthGuard and RoleGuard.

Designed Guard	Status	Implementation Notes
AuthGuard	Implemented	src/app/guards/auth.guard.ts — functional CanActivateFn guard. Checks AuthService.isLoggedIn(). Redirects unauthenticated users to /login. Applied to both /dashboard and /student-dashboard. Matches design intent exactly.
RoleGuard	Implemented	src/app/guards/role.guard.ts — functional CanActivateFn guard. Reads route.data['role'], compares to AuthService.getRole(). Redirects wrong-role users to their correct dashboard. Matches design intent exactly.

Summary: Both designed guards are fully implemented.

1.4 Angular Modules

The Final Design specifies four NgModules structuring the application: AppModule, CoreModule, StudentModule, AdminModule.

Designed Module	Status	Implementation Notes
AppModule	Implemented	src/app/app.module.ts — root @NgModule importing AppComponent, CoreModule, StudentModule, and AdminModule. Note: the application is bootstrapped via bootstrapApplication() in main.ts (Angular 17+ standalone pattern). AppModule exists as the NgModule-based structural equivalent and groups all feature modules.
CoreModule	Implemented	src/app/core.module.ts — @NgModule declaring the shared infrastructure services (AuthService, ApiService, NotificationService, ProgressService).
StudentModule	Implemented	src/app/modules/student.module.ts — @NgModule grouping all student-facing components: StudentDashboardComponent, ApplicationComponent, ContractComponent, ComplaintComponent, DocumentsComponent, ReceiptComponent, ProgressComponent.
AdminModule	Implemented	src/app/modules/admin.module.ts — @NgModule grouping all admin-facing components: DashboardComponent and admin feature components.

Summary: All 4 designed NgModules are implemented. They coexist alongside the bootstrapApplication() standalone pattern used at runtime. The modules serve as the NgModule-based architectural grouping as specified in the design, even

though Angular 21's standalone approach is the active bootstrap mechanism.

2. Data Model — Entities and Relationships

2.1 Entity Compliance

The Final Design specifies eight core entities: User, StudentProfile, DormApplication, Contract, RentPayment, Complaint, Report, FileMetadata.

Designed Entity	Status	Implementation Notes
User	Implemented	users table: userid, name, email, passwordhash, role ENUM('STUDENT','ADMIN'), dormitoryid. Matches all designed attributes. dormitoryid is an addition beyond the single-dormitory design scope.
StudentProfile	Implemented	studentprofiles table: profileid, userid, roomid, phone, studentidnumber, course, university, application_status. Maps to designed attributes (academicDetails, roomInformation, applicationStatus).
DormApplication	Implemented	dormapplications table: applicationid, userid, submissiondate, status ENUM('PENDING','ACCEPTED','REJECTED'), assignedroomid. Matches design. Additions: applicationtype, remarks, dormitoryid.
Contract	Implemented	contracts table: contractid, userid, roomid, startdate, enddate, status ENUM('ACTIVE','EXTENDED','TERMINATED'). Matches design. Additions: monthlyrent, dueday, generateddocfileid, signeddocumentfileid, terminationreason.
RentPayment	Implemented	rentpayments table: paymentid, userid, month, amount, receiptpath, createdat. Matches design fields. Addition: verificationstatus ENUM('PENDING_VERIFICATION','VERIFIED','REJECTED').
Complaint	Implemented	complaints table: complaintid, userid, description, status ENUM('SUBMITTED','INPROGRESS','RESOLVED'), createdat. Matches design exactly.
Report	Implemented	reports table: reportid, userid, filepath, createdat. Matches core design fields. Additions: status, progressid, reportsource ENUM('ADMIN','STUDENT').
FileMetadata	Implemented	filemetadata table: fileid, filepath, filetype, uploaddate. Matches design. Additions: applicationid, user_id for ownership tracking.
dormitories (not in design)	Added	Multi-dormitory support table. Contains name, address, contactemail, contactphone, max_capacity, notification preference flags. Not in original single-dormitory design.
rooms (not in design)	Added	rooms table: roomid, roomnumber, floor, type, status ENUM('occupied','vacant','maintenance'), resident_id. Rooms were implicit in the design but not a separate entity.
termination_requests (not in design)	Added	Student-initiated contract termination request table. Not in original design.
notifications (not in design)	Added	Real-time SSE notification storage. Not in original design.
progress (not in design)	Added	Backend long-running task progress tracking in DB. Design tracked progress in-memory; DB table is an addition.

admin_profiles (not in design)	Added	Admin profile table: userid, dormitoryid, position, phone. Not in original design. Added for admin profile management.
-----------------------------------	-------	--

2.2 Relationship Compliance

Designed Relationship	Status	Notes
User has exactly one role (STUDENT or ADMIN)	Implemented	role ENUM column on users. Enforced by requireRole() middleware on all protected endpoints.
User (STUDENT) has 0..1 StudentProfile	Implemented	studentprofiles.userid FK. One profile per student, inserted at registration.
User (STUDENT) can submit many DormApplications	Implemented	dormapplications.userid FK. Multiple applications per student supported.
User (STUDENT) has at most one active Contract	Implemented	Enforced via transactional PATCH /api/applications/:id/status — checks for existing active contract before creating a new one.
User (STUDENT) has many RentPayments	Implemented	rentpayments.userid FK. Multiple payments per student allowed.
User (STUDENT) has many Complaints	Implemented	complaints.user_id FK. Multiple complaints per student allowed.
User can generate many Reports	Implemented	reports.user_id FK. Multiple reports per user supported.
DormApplication, Contract, RentPayment reference FileMetadata	Implemented	filemetadata.applicationid for application documents. contracts.generateddocfileid and signeddocumentfileid FK to filemetadata. rentpayments.receipt_path stores receipt reference.

Summary:All 8 designed relationships are correctly implemented.

3. Behavioral Design — Sequence Diagrams

The Final Design includes six sequence diagrams covering core workflows.

3.1 Login Scenario

Design:LoginComponent !' AuthenticationService !' backend !' role check via AuthGuard/RoleGuard !' redirect to correct dashboard.

Status	Implemented
Compliance	The implemented login flow matches the design exactly. LoginComponent calls AuthService.login() !' POST /api/login !' JWT stored in localStorage !' authGuard and roleGuard applied via canActivate on both dashboard routes !' redirected to /dashboard (ADMIN) or /student-dashboard (STUDENT).

3.2 Application Submission Scenario

Design:ApplicationComponent !' FileService (upload docs) !' ApplicationService (submit) !' confirmation displayed.

Status	Implemented
Compliance	The workflow is fully functional and the designed components/services are now present. ApplicationComponent exists as a standalone component receiving data via @Input. ApplicationService and FileService both exist as named domain services. The orchestration (wiring inputs/outputs and calling the services) is handled by StudentDashboardComponent. The sequence of actions — file upload !' application submission !' confirmation display — matches the design.

3.3 Application Review & Decision Scenario

Design:AdminDashboardComponent !' ApplicationService !' backend retrieves pending apps !' admin accepts/rejects !' persisted.

Status	Implemented
Compliance	Admin dashboard fetches applications via ApiService.getApplications() (also available through ApplicationService). Accept flow opens acceptDialog modal !' PATCH /api/applications/:id/ status with transactional room assignment. Reject flow opens rejectDialog !' same endpoint with REJECTED status and remarks. Matches design workflow.

3.4 Complaint Submission & Processing Scenario

Design:ComplaintComponent !' ComplaintService !' backend stores complaint !' admin manages via AdminDashboardComponent !' ComplaintService updates status.

Status	Implemented
Compliance	ComplaintComponent exists as a standalone component emitting submitComplaint events. ComplaintService exists and exposes submitComplaint() and updateComplaintStatus(). Admin manages complaints via PATCH /api/complaints/:id/status. Workflow matches the design.

3.5 Rent Payment Receipt Upload & Download Scenario

Design:ReceiptComponent !' FileService (upload) !' backend stores receipt !' ReceiptService creates payment record !' students/ admins can download.

Status	Implemented
Compliance	ReceiptComponent exists as a standalone component emitting submitReceipt events. ReceiptService exists and exposes recordPayment(), getPayments(), verifyPayment(), and downloadReceipt(). Admin downloads via GET /api/payments/:id/receipt. Workflow matches the design.

3.6 Rent Payment Report Generation with Progress Scenario

Design: Dashboard !' ReportService !' backend starts long-running task !' ProgressService polls progress !' ProgressComponent displays indicators !' download when complete.

Status	Implemented
Compliance	This workflow matches the design most closely and all designed elements are now present. ReportService exposes generateReport() and downloadReport(). ProgressService owns startPolling() (600ms interval) and stopPolling(). ProgressComponent exists as a standalone component with @Input() percentage and @Input() status bindings. The async backend pattern (setImmediate !' generatePdfReport() !' progress milestones) is unchanged.

Summary: All 6 designed workflows are fully implemented end-to-end with the designed structural components and services now present.

4. Backend Design

4.1 Technology Compliance

Design Element	Status	Implementation Notes
Node.js + Express.js REST API	Implemented	server.js — single-file Express app on port 3000. 40+ REST endpoints with authMiddleware and requireRole(). Matches design.
MySQL relational database	Implemented	mysql2 driver. dorm_management database. All 8 designed tables implemented plus 6 additional tables. Matches design.
JWT authentication + role-based access	Implemented	jsonwebtoken library. authMiddleware decodes and attaches req.user. requireRole() enforces role-based access. Matches design.
Async long-running report generation	Implemented	POST /api/reports uses setImmediate() for non-blocking PDF generation. generatePdfReport() updates progress table at 25/50/75/100% milestones. Matches design.
Progress tracking via API	Implemented	GET /api/reports/:id/progress reads progress table and returns { percentage, status }. Polled by frontend every 600ms. Matches design.

File upload and download	Partially Implemented	Multer disk storage to /uploads/ subdirectories. All file types (contracts, receipts, application docs) served via download endpoints. Design specified Google Firebase Storage — local disk used instead. Functionally equivalent for local dev; not production-ready per design.
Google Firebase Storage	Not Implemented	Design specified Firebase for file storage. Local disk storage used throughout. This is the primary backend deviation from the design.

5. Design Compliance Summary

5.1 Compliance by Area

Design Area	Designed	Implemented	Partially	Not Implemented
Angular Components	11	11	0	0
Business Services	8	7	1	0
Infrastructure Guards	2	2	0	0
Angular Modules (NgModules)	4	4	0	0
Data Entities	8	8	0	0
Entity Relationships	8	8	0	0
Behavioral Workflows	6	6	0	0
Backend Technologies	6	5	1	0

5.2 Key Deviations from Design

- Domain Services as Thin Wrappers — The design specifies 7 domain-specific services (ApplicationService, ContractService, ComplaintService, ReceiptService, FileService, ReportService, ProgressService) encapsulating business logic. All seven exist as separate files and expose the correct domain method names. However, their implementations are thin facades that delegate to the monolithic ApiService, which holds all actual HTTP call logic. The intended domain-separation of concerns is partially achieved (named boundary classes exist) but the HTTP implementation is still centralized in ApiService.
- Presentational Feature Components — The design specifies 7 standalone feature components (ApplicationComponent, ContractComponent, ComplaintComponent, DocumentsComponent, ReceiptComponent, ProgressComponent, NavMenuComponent). All 7 exist and follow the smart/dumb component pattern: they are presentational components receiving data via @Input and emitting events via @Output. Orchestration logic (service calls, state management) remains in the two dashboard components. The structural elements exist; the full component-owns-its-domain intent of the design is approximated rather than fully realized.
- NgModules alongside Standalone Bootstrap — The design specifies AppModule, CoreModule, StudentModule, AdminModule. All four NgModules now exist as files grouping the correct components and services. However, the application is bootstrapped via bootstrapApplication() (Angular 17+ standalone pattern) rather than via AppModule. The NgModules serve as the NgModule-based architectural document as designed but are not used by the runtime bootstrap.
- Firebase ! Local Disk Storage — The design specified Google Firebase Storage for all user-uploaded files. Local disk storage via Multer (/uploads/ directory) is used throughout. Functionally equivalent in a local development environment; not production-ready as specified.

- Extra Entities Beyond Design Scope — Six tables added beyond the designed schema: dormitories, rooms, terminationrequests, notifications, progress, adminprofiles. These are driven largely by the multi-dormitory scope creep and the real-time notification system — both features that go beyond what the requirements and design specify.
-

5.3 Correctly Implemented Design Elements

The following design elements are implemented correctly and match the Final Design specification:

- All 11 designed Angular components (4 core + 7 feature components)
 - All 7 domain services with correct names and method signatures
 - All 4 NgModules grouping the correct components and services
 - All 8 core data entities with correct attributes and types
 - All 8 designed entity relationships
 - AuthGuard and RoleGuard — implemented exactly as designed (functional CanActivateFn guards)
 - JWT authentication with role-based access control on all endpoints
 - Two-role system (STUDENT / ADMIN) with correct access separation
 - Async long-running report generation (setImmediate) with DB progress tracking
 - Frontend progress polling (ProgressService) and visual progress bar (ProgressComponent)
 - Dialog-based confirmations for complaint submission (R22) and application decisions (R23)
 - Router-based menu navigation using Angular child routes
 - File upload and download for all three categories (contracts, application docs, receipts)
 - All 6 behavioral workflows function end-to-end as described in the sequence diagrams
-

5.4 Recommendations

- Wire Feature Components into Dashboards — The 7 feature components exist as presentational components but are not yet wired as child components inside the dashboard templates. Connecting them would complete the smart/dumb component architecture: dashboards become orchestrators passing data via @Input and handling @Output events from the feature components.
 - Move HTTP Logic into Domain Services — The 7 domain services delegate to ApiService. A further refinement would be to move the actual HTTP call implementations from ApiService into each domain service. ApiService could then either be removed or retained as a low-level HTTP utility.
 - Activate NgModules as Runtime Bootstrap — Migrate from bootstrapApplication() to platformBrowserDynamic().bootstrapModule(AppModule) to fully activate the NgModule-based structure as designed. Both patterns are supported in Angular 21.
 - Firebase Storage — If the project is intended for production deployment, replace Multer disk storage with Firebase Storage as originally designed. For local/academic use, the current implementation is adequate.
-

